

NAME: Samiksha Hajare

PRN:202501070144

BRANCH : ENTC

DIVISION :ET21

PRACTICAL EDS :

1.1.1 Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

where:

is the mass of the object (in kilograms).

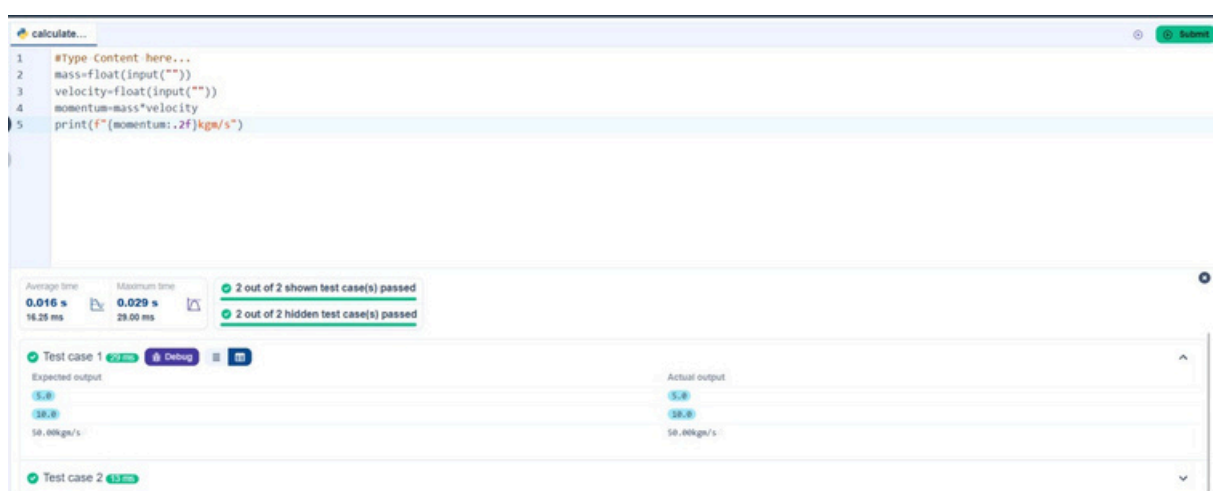
is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).



```
1 #Type Content here...
2 mass=float(input(""))
3 velocity=float(input(""))
4 momentum=mass*velocity
5 print(f"momentum: .2f)kgm/s")
```

Average time: 0.016 s, Maximum time: 0.029 s, 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case	Expected output	Actual output
Test case 1	5.0 18.0 50.00kgm/s	5.0 18.0 50.00kgm/s
Test case 2		

- 1.1.2 practical
Write a Python program that accepts an integer as input. Depending on the number of digits in .
-
- **Constraints:**
- $1 \leq n \leq 999$
-
- **Input Format:**
- The input consists of a single integer .
-
- **Output Format:**
- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

The screenshot shows a web-based code editor interface for DE TANTRA. The editor contains a Python program that takes an integer input and performs different operations based on the number of digits. The program is as follows:

```

1 #write your code here...
2 import math
3 n=int(input(""))
4 if 1<=n<=9 :
5     print(n*n)
6 elif 10<=n<=99 :
7     print(f"{math.sqrt(n):.2f}")
8 elif 100<=n<=999 :
9     print(f"{n**(1/3):.2f}")
10 else :
11     print("Invalid")
12

```

Below the code editor, the test results are displayed. The average time is 0.006 s and the maximum time is 0.010 s. The program has passed 4 out of 4 shown test cases and 3 out of 3 hidden test cases. The test cases are listed as follows:

Test Case	Status
Test case 2	Passed
Test case 3	Passed
Test case 4	Passed

1.1.3

Write a Python program to calculate number of days between two dates.

Input Format:

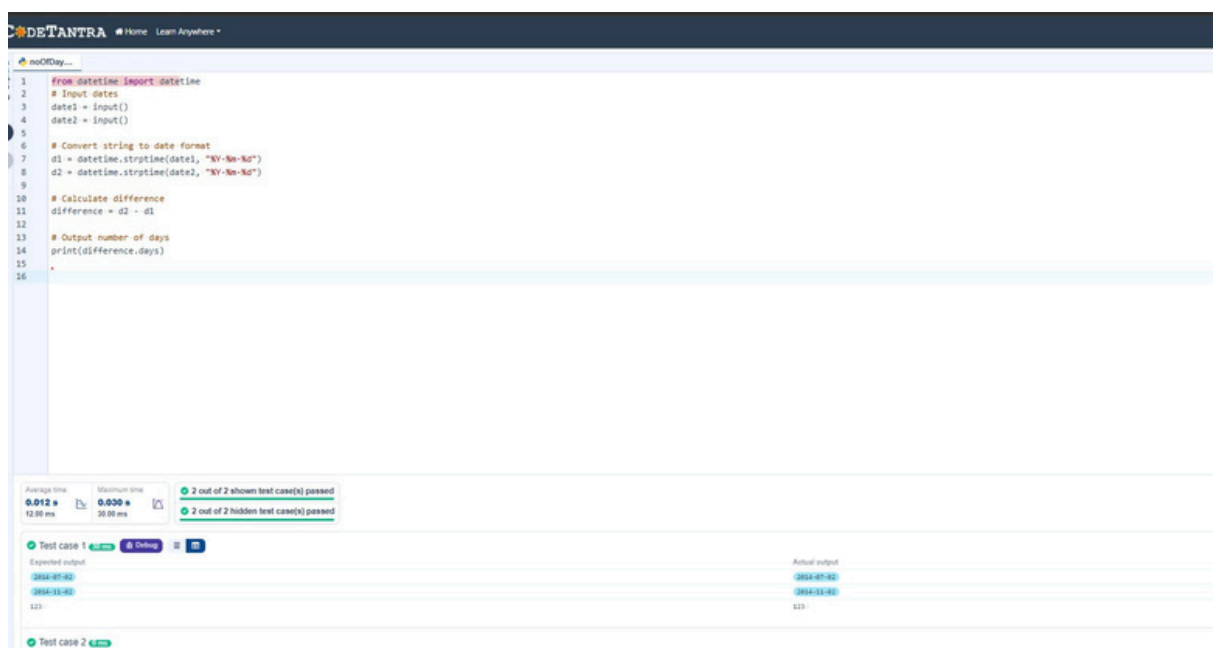
- The first line contains the first date in the format .
- The second line contains the second date in the format .

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.



```
1 from datetime import datetime
2 # Input dates
3 date1 = input()
4 date2 = input()
5
6 # Convert string to date format
7 d1 = datetime.strptime(date1, "YY-MM-KK")
8 d2 = datetime.strptime(date2, "YY-MM-KK")
9
10 # Calculate difference
11 difference = d2 - d1
12
13 # Output number of days
14 print(difference.days)
15
16
```

Average time: 0.012 s, Maximum time: 0.020 s, 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed

Test case 1	Expected output	Actual output
	2024-07-02	2024-07-02
	2024-11-02	2024-11-02
	123	123

Test case 2

1.1.4

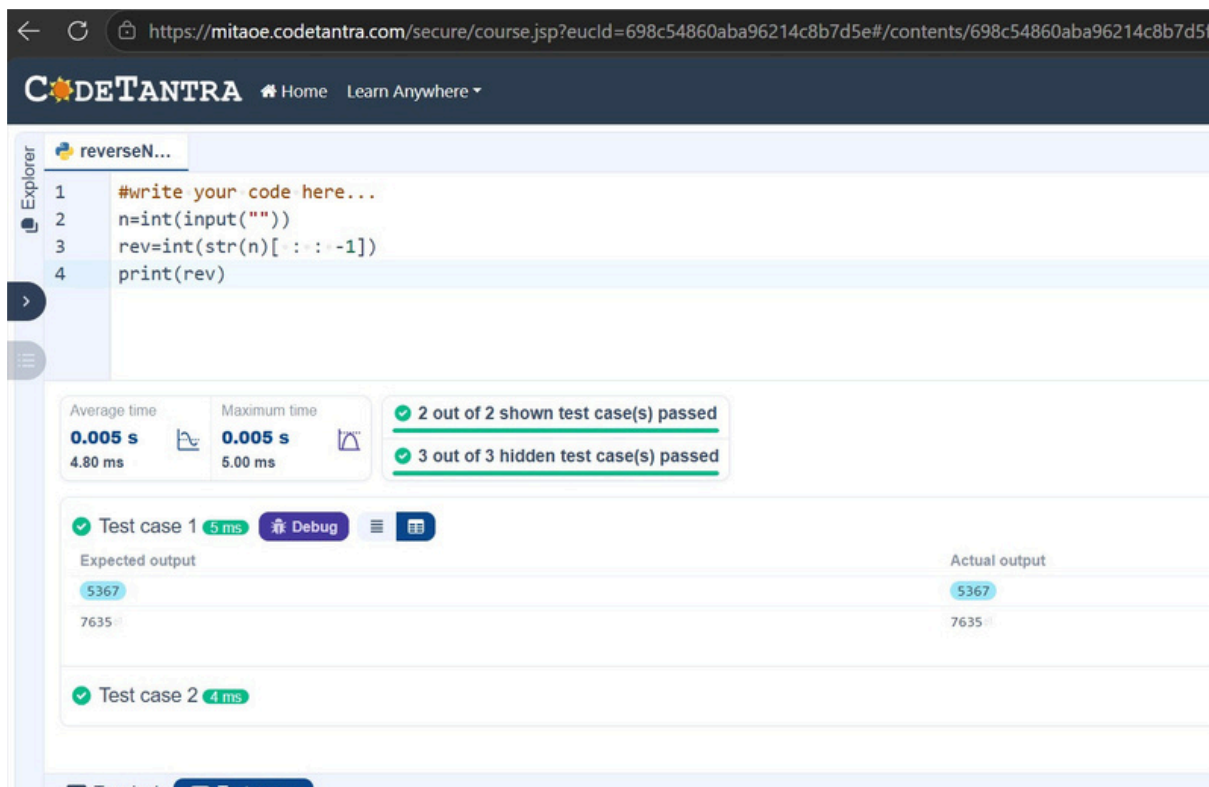
Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.



1.1.5

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

The screenshot shows a web browser with the URL <https://mitaoe.codetantra.com/secure/course.jsp?eucid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c5486...>. The page header includes the CodeTANTRA logo, a home icon, and the text "Learn Anywhere". The main content area is titled "multiplica..." and contains a Python code editor with the following code:

```
1 #write your code here...
2 num=int(input(""))
3 for i in range(1,11):
4     print(f"{num} x {i} = {num*i}")
5
6
7
```

Below the code editor, the execution results are displayed. The average time is 0.011 s (11.25 ms) and the maximum time is 0.019 s (19.00 ms). The test results show that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The output of the program for input 8 is shown as follows:

Input	Output
8	8 x 1 = 8 8 x 2 = 16 8 x 3 = 24 8 x 4 = 32 8 x 5 = 40 8 x 6 = 48 8 x 7 = 56 8 x 8 = 64

1.2.1

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.

- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

write your code here

```
n=int(input())
```

```
marks = list(map(int, input().split()))
```

```
failed = False
```

```
for i in marks:
```

```
if i < 40 :
```

```
failed=True
```

```
break
```

```
iffailed:
```

```
print("Fail")
```

```
else :
```

```
agg=sum(marks)/n
```

```
print("Aggregate Percentage: "{agg:.2f}")
```

```
if agg > 75 :
```

```
print("Grade: Distinction")
```

```
elif agg >=65 :
```

```
print("Grade: First Division")
```

```
elif agg >=50:
```

```
print("Grade: Second Division")
```

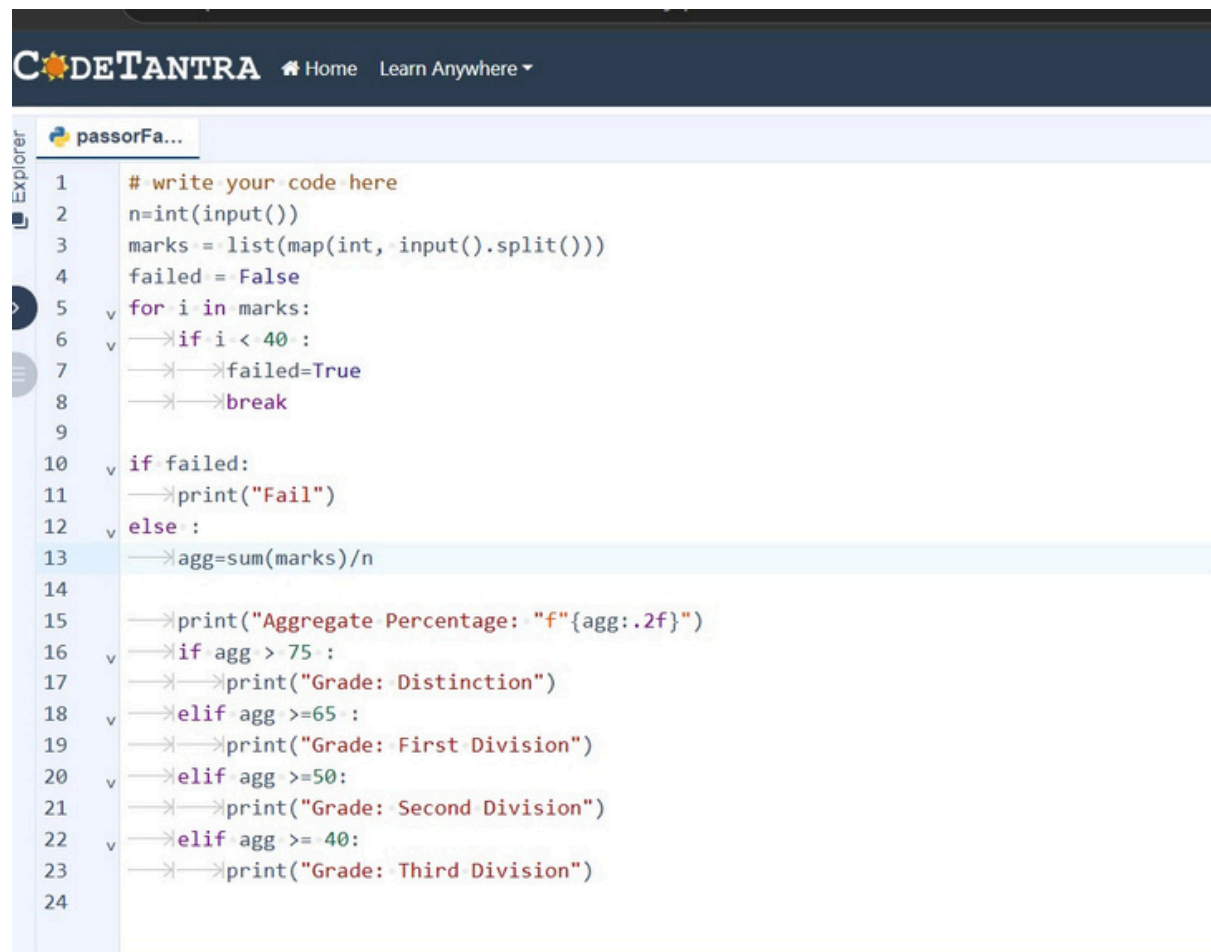
```
elif agg >= 40:
```

```
print("Grade: Third Division")
```

Translation Failed

Reason for late submission

Please enter at least 15 characters



The screenshot shows a web-based IDE interface for CodeTANTRA. The top navigation bar includes the logo and links for 'Home' and 'Learn Anywhere'. Below the navigation bar, a file explorer on the left shows a file named 'passorFa...'. The main editor area displays Python code for calculating a grade based on an aggregate percentage. The code includes comments, variable declarations, input handling, and conditional logic for different grade ranges. Line numbers 1 through 24 are visible on the left side of the code editor.

```
1  # write your code here
2  n=int(input())
3  marks = list(map(int, input().split()))
4  failed = False
5  for i in marks:
6      if i < 40 :
7          failed=True
8          break
9
10 if failed:
11     print("Fail")
12 else :
13     agg=sum(marks)/n
14
15     print("Aggregate Percentage: "{agg:.2f}")
16     if agg > 75 :
17         print("Grade: Distinction")
18     elif agg >=65 :
19         print("Grade: First Division")
20     elif agg >=50:
21         print("Grade: Second Division")
22     elif agg >= 40:
23         print("Grade: Third Division")
24
```

CodeTANTRA Home Learn Anywhere

passorFa...

```
1 # write your code here
2 n=int(input())
3 marks = list(map(int, input().split()))
4 failed = False
5 for i in marks:
```

Average time: 0.009 s (9.20 ms) Maximum time: 0.013 s (13.00 ms)

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 (7 ms) Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First Division	Grade: First Division

Test case 2 (10 ms)

1.2.2

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

```
1 def fibonacci(n):
2
3     # write the code..
4     if n==1:
5         return 0
6     elif n==2:
7         return 1
8     else:
9         return fibonacci(n-1)+fibonacci(n-2)
10
11
12 n = int(input())
13 for i in range(1, n + 1):
14     print(fibonacci(i), end=" ")
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Average time: 0.014 s, 14.25 ms | Maximum time: 0.039 s, 39.00 ms | 2 out of 2 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1: 5 ms | Debug | Expected output: 0 1 1 2 3

Test case 2: 5 ms

Terminal | Test cases

1.2.3

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

The screenshot shows a Python IDE with a file named 'rightangl...'. The code is as follows:

```
1 #Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     print("* "*i)
```

Below the code, the execution results are displayed:

Average time	Maximum time	Test Results
0.006 s 5.67 ms	0.008 s 8.00 ms	2 out of 2 shown test case(s) passed 4 out of 4 hidden test case(s) passed
Test case 1 7 ms		
Test case 2 6 ms		

1.2.4

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

 numberP...

```

1  n = int(input())
2
3  # Loop for rows
4  for i in range(1, n + 1):
5      # Loop for printing numbers
6      for j in range(1, i + 1):
7          print(j, end=" ")
8      print()

```

 numberP...

```

1  n = int(input())
2
3  # Loop for rows
4  for i in range(1, n + 1):
5      # Loop for printing numbers

```

Average time

0.009 s

9.00 ms



Maximum time

0.018 s

18.00 ms


 2 out of 2 shown test case(s) passed

 2 out of 2 hidden test case(s) passed

 Test case 1 **6 ms**
 Test case 2 **18 ms**